XPoster — Unified Application Workflow and Design Plan

Authoritative merged specification | Application, Python engine, JSON-Lines protocol, idle handshake, and implementation verification

DOCUMENT STATUS: DESIGN BASELINE — not an assertion that the application has been implemented or that live X posting has passed. This specification replaces conflicting older proposals for the purpose of future implementation.

Executive summary

XPoster is a personal-use Windows application with a Windows PowerShell 5.1 WinForms interface, a short-lived Python 3.13 X API engine, and local TXT/media/CSV history. PowerShell controls the user interface, application state, local export, process lifecycle, and diagnostics; Python controls request validation, OAuth 2.0 User Context, sequential media upload, and final post creation. No browser automation, persistent Python daemon, local web service, or automatic posting is part of this design.

```
USER / GUI  →  XPoster.ps1  →  XPoster.py  →  X API
                  ↘️ local TXT + media + CSV history
IDLE ONLY: PS1 timer (10 s) → short-lived Python health check → X → PS1
```

CRITICAL ORDER: A manual Post action first commits a local archive record and managed media copies; only after that succeeds does PowerShell start the Python posting engine. Python reports actual acknowledged uploaded byte counts, and PowerShell computes the progress percentage. A separate 10-second idle health check is silent on success and warns/logs on failure.

1. Scope, authority, and evidence

1.1 Source precedence and conflict resolution

Priority	Authority	Interpretation
1	Latest explicit user decisions	Overrides older examples and defaults.
2	Merge conversation and later decisions	Resolves previous open questions; including DEBUG, bytes, OAuth2, Title, media copies, and handshake.
3	LocalTwitterPoster_ProgressV2	Approved names, state, paths, export-before-Python order, GUI, history and Repost.
4	Person 1 specification	Application behavior, user workflow, archive and verification checklist.
5	Person 2 specification	Implementation safeguards, asynchronous process design, state machine, mock harness, fault handling.
6	Older protocol and prototype drafts	Background only where consistent with the higher authorities.

The accompanying Python workflow and Idle Handshake Work Plan are mandatory specialist specifications. They supplement the above authority order; their older Percent-based progress examples are superseded by the user-approved byte-count contract. The merged design does not silently elevate untested engineering assumptions into approved facts.

1.2 Source inventory and scope

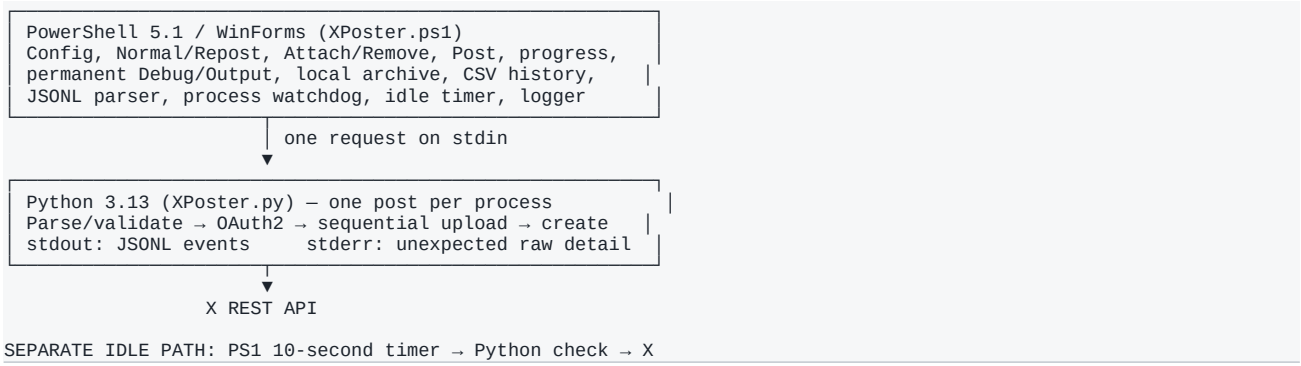
Source	Contribution
GPT_Understanding.txt	Deliverable scope, precedence, Word/PDF parity, no timestamps or mixed-language text.
LocalTwitterPoster_ProgressV2(1).txt	Final application control flow, GUI state, paths, names, request schema, Stage 1 gate.
Person1XPoster_Python_Workflow_Function_Design(2).pdf	Application design fidelity, history/repost and original review checklist.

Source	Contribution
Person2XPoster_Python_Workflow_Function_Design(2).pdf	Nonblocking process pump, explicit state machine, error/timeout logic, Python mock tests.
Person1Person2MergeIdea(2).txt	Merge rationale, engineering defects and mismatches to eliminate.
MergeConvoAndMissingDetails.txt	Latest precedence decisions and DEBUG protocol correction.
XPoster_Python_Workflow_Function_Design_COMPLETE(1).pdf	Python flow and function contracts, security, terminal event rules.
XPoster — Idle Handshake Work Plan.txt	Idle timer, short-lived Python check, silent success, warning/dedup, suspend/resume.
LocalTwitterPosterDesignV2.txt and DebugLogReadMe.txt	CSV/Excel/TXT intent, stage gate and exact existing logger API.

1.3 Classification labels

LOCKED = approved user behavior or exact name. ENGINEERING DESIGN = chosen implementation mechanism that preserves locked behavior and must be validated on the target Windows machine. VERIFY BEFORE LIVE USE = endpoint, credential, account, timing, or API behavior not proven by the supplied evidence. DEFERRED = expressly outside the current implementation scope.

2. System architecture and boundaries



PowerShell owns	Python owns
GUI, user actions, form and posting state	Request parsing and strict validation
Config bootstrap and shared logger	Python-relevant config and credentials
TXT/managed media/history export BEFORE Python	OAuth2 user-context session and token handling
Process start, stdout/stderr pump, exit and watchdogs	Sequential X media upload and processing checks
RequestId/event/state validation	Structured DEBUG, PROGRESS, MEDIA_COMPLETE, terminal result
Byte-to-percent math, status and progress bar	Final create-post call and remote IDs
Repost JSON import and manual review	Read-only X handshake operation on explicit invocation

A posting process receives exactly one request, emits at most one terminal event, and exits. A health-check process is a different, short-lived mode. There is no permanent Python worker, local HTTP port, socket server, or request queue.

3. Fixed files, configuration, and credentials

3.1 Existing physical layout

```
Z:\InsPro\Tools\
  Py\XPoster.py
  Ps1\XPoster.ps1
  Ps1\DebugLog.ps1
  Ps1\ReadMe\DebugLogReadMe.txt
  Data\X_Post\
    Operational\X_Config.txt
    Operational\X_Keys.txt
    TextFiles\
    MediaFiles\
    ExcelFile\
  Logs\XPoster\<per-run-log>
```

The folder listing X_PostFolderStructure.txt is reference material only, not an application input. ExcelFile is reserved for the CSV history and a future Excel browser/template.

3.2 Exact X_Config.txt contract — preserve key names and values

```
PythonPath=C:\Python\Python313\python.exe
PythonScriptPath=Z:\InsPro\Tools\Py\XPoster.py
DataRootPath=Z:\InsPro\Tools\Data\X_Post
OperationalPath=Z:\InsPro\Tools\Data\X_Post\Operational
TextFilesPath=Z:\InsPro\Tools\Data\X_Post\TextFiles
MediaFilesPath=Z:\InsPro\Tools\Data\X_Post\MediaFiles
ExcelFilePath=Z:\InsPro\Tools\Data\X_Post\ExcelFile
KeysFilePath=Z:\InsPro\Tools\Data\X_Post\Operational\X_Keys.txt
DebugLogReadMePath=Z:\InsPro\Tools\Ps1\ReadMe\DebugLogReadMe.txt
DebugLogFunctionPath=Z:\InsPro\Tools\Ps1\DebugLog.ps1
DebugLogMode=Per run
DebugLogOutputPath=Z:\InsPro\Tools\Logs\XPoster\xxxxxxx.log
```

ReadConfig splits on the FIRST equals sign and preserves literal values (notably "Per run"). PowerShell needs one bootstrap location for X_Config.txt before ReadConfig; all other paths are obtained from Config. Python receives the config path via --config, not a second hard-coded location. The illustrative xxxxxxx.log is NOT the actual logger-generated filename; Get-DebugLogPath provides the actual per-run path.

3.3 X_Keys.txt — selected field names

```
AUTH_MODE=OAuth2
CLIENT_ID=<local value>
CLIENT_SECRET=<local value>
ACCESS_TOKEN=<local value>
REFRESH_TOKEN=<local value>
ACCESS_TOKEN_EXPIRES_AT=<expiry representation>
```

OAuth 2.0 User Context is the chosen design, not OAuth 1.0a. Exact token expiry representation, permitted refresh scope, OAuth2 confidential/public-client behavior, and X endpoint support remain VERIFY BEFORE LIVE USE. No password or authenticator code is used by normal API calls. Token values, Authorization headers, full key file and raw post bodies must never appear in logs or design artifacts. Refresh must preserve credential integrity and avoid logging secrets; exact write/refresh method requires verification.

3.4 Existing debug logger — do not replace

```
dot-source: Config.DebugLogFunctionPath
Initialize-DebugLog -Version 1 -PerRun
Write-DebugLine <Message> [INFO | WARN | ERROR]
Get-DebugLogPath
```

The exact existing logger supports INFO, WARN and ERROR; there is no approved Write-DebugLog or Source parameter. Prefix messages with [PYTHON] or [PS1] in the Message text. Map protocol Level WARNING to logger WARN and controlled ERROR-DIAGNOSTIC to ERROR if appropriate. The permanent GUI Debug/Output panel displays controlled diagnostics even though DebugLog.ps1 handles the file. A logging failure does not silently become posting success.

4. GUI and startup workflow

4.1 Normal and Repost startup

```
MainController → ReadConfig → initialize logger
               → GuiController → DetermineStartupMode
                    ├─ Normal: blank PostText, no selected media
                    └─ Repost: validate JSON, resolve TXT/media, preload
               → InitializeGui → UpdatePostButtonState
               → idle GUI (health timer enabled)
```

Repost JSON is an application startup contract. The Excel workbook/button, VBA, XML/XSL, command-line transport, and workbook template are DEFERRED. DetermineStartupMode and InitializeGui own the preload behavior; do not reintroduce an independent LoadRepostData function as a locked requirement. Repost NEVER triggers Post automatically.

4.2 Visible controls and invariants

Control/state	Required behavior
PostText	Multiline editor; text is preserved exactly when sent to Python.
Attach Media	File picker stores paths only; no upload until Post.
SelectedMedia[]	Up to four visible entries with filenames; independent of text-entry order.
RemoveMedia	Removes selected entry, refreshes list and button state.
Post button	Disabled if both text and media are empty; enabled if either exists; disabled during posting.
Progress	One bar and file label; switches for each sequential media item.
Debug / Output	Permanently visible; controlled Python DEBUG and errors, PS1 process warnings, post result.
Repost preload	Shows saved text/media for user edits; requires manual Post.
Posting lock	No double-click publishing or mutation of the in-flight request.

Text-only and media-only posts are valid. A post with neither text nor media is invalid. Media combinations and X constraints must be verified against the selected endpoint and account; do not assume that any arbitrary four-file mix is supported.

5. Authoritative end-to-end posting workflow

```
USER presses Post
  → verify nonempty post, no active post, and unknown-outcome rule
  → freeze finalized text + selected-media snapshot
  → set PostInProgress = true; suspend idle handshake
  → ExportPostRecord (local transaction)
       ├─ generate unique RequestId and Title
       ├─ save full UTF-8 text to TextFiles/<RequestId>.txt
       ├─ copy each selected file to MediaFiles/<RequestId>/
       └─ append history row with blank PostURL/Timestamp
  → IF EXPORT FAILED: stop; report error; do NOT start Python
  → RunPythonEngine using archived media paths
  → receive streamed READY / TEXT_COMPLETE / PROGRESS / ...
  → on SUCCESS: update SAME history row PostURL and Timestamp
  → on ERROR: preserve archived record and blank publication fields
  → finish/cleanup process and restore GUI state
  → PostInProgress = false; resume 10-second idle checks
```

The local record is saved even if X fails. The TXT path / PostRecordPath is the stable internal identity for updating the SAME eight-column CSV row without inventing a ninth RequestId or Status column. Local export failure is a pre-publication stop condition. Do not append extra Pending/Failed/Published status columns or multiple history rows for one attempt. DebugLog records the detailed outcome. If an ambiguous remote create outcome occurs, preserve the record and require manual verification on X before a new Post attempt.

5.1 Local archive transaction and recovery

ENGINEERING DESIGN: use a per-attempt staging path and atomic or carefully ordered file writes where practical. Do not publish to X unless the TXT, required media copies, and history entry are all committed and readable. If local export partially fails, report it and

leave a diagnosable local state rather than silently discarding user content. Use try/finally-style cleanup so PostInProgress and the idle timer return to a valid state even when export fails before Python starts. No automatic retry of the remote post. Preserve original user-selected paths in GUI state; the Python request uses the managed copies.

5.2 Title generation — locked

If Text has a non-empty line, Title is its first non-empty line truncated to 80 characters. Otherwise, Title is the first media filename without its extension. Empty title cannot occur in a valid post. This is a derived history label, not a new required GUI field. Do not rewrite the underlying post text.

6. History, CSV/Excel, and Repost boundary

6.1 Exact history columns

```
Title | Text | PostURL | Timestamp | Media 1 | Media 2 | Media 3 | Media 4
```

Column	Value / display
Title	Auto-derived readable title.
Text	Excel-visible hyperlink label "Display Text" pointing to the saved TXT.
PostURL	Published X URL on SUCCESS; blank before success or after a known failure.
Timestamp	Actual publication time on SUCCESS; blank before success or after a known failure.
Media 1–4	Each occupied cell links to a managed local file and displays its filename; unused cells blank.

The authoritative local index is a CSV intended for browsing in Excel. An Excel-compatible hyperlink representation may be used, but CSV has no native rich hyperlink type; formula escaping, spreadsheet formula injection protection, and round-trip path resolution must be tested. Do not invent extra CSV columns or claim that an Excel template exists. If the CSV link format cannot safely satisfy both display and machine-readable Repost, keep the eight-column presentation requirement and resolve the exact template mechanism in the DEFERRED Excel phase.

6.2 Repost JSON application contract

```
Illustrative startup payload (exact launcher deferred):
{
  "Mode": "Repost",
  "RepostData": {
    "TextPath": "<saved local TXT path>",
    "MediaPaths": ["<saved local media path>"]
  }
}
```

PowerShell validates paths, reads TXT, loads up to four media into SelectedMedia, refreshes the GUI and Post-button state, and waits for manual review. When the user clicks Post, create a NEW RequestId and NEW local archive record. The JSON structure is a design interface; exact transport and Excel implementation remain deferred.

7. PowerShell function tree and contracts

```
MainController
├─ ReadConfig
├─ GuiController
│  ├─ DetermineStartupMode
│  ├─ InitializeGui
│  ├─ AttachMedia
│  ├─ RemoveMedia
│  ├─ UpdatePostButtonState
│  └─ UpdateGui
├─ StartPost
│  ├─ ExportPostRecord
│  ├─ RunPythonEngine
│  └─ HandlePythonResponse
└─ GuiController
   └─ UpdateUploadProgress

Additional helper operations (engineering implementation):
  IdleHandshakeTimer / RunHealthCheck / HandleHealthResult
  ProcessPump / ProtocolValidator / ProcessWatchdog / DebugOutput
```

Locked function	Inputs / state	Contract
MainController	Config, GUI and process state	Application orchestration, lifecycle and cleanup.
ReadConfig	ConfigPath	Read/validate exact key=value config; return Config.
GuiController	StartupMode, RepostData	Own GUI action dispatch and state.
DetermineStartupMode	startup JSON or Normal	Return Normal/Repost and validated preload metadata.
InitializeGui	StartupMode, RepostData	Create form, bind callbacks, preload when Repost.
AttachMedia	SelectedMedia	Add file path if capacity permits; no upload.
RemoveMedia	MediaIndex	Remove selected media; refresh list and state.
UpdatePostButtonState	PostText, SelectedMedia	Enforce empty/in-progress button rules.
UpdateGui	GuiState	Render status, messages and controls on GUI thread.
UpdateUploadProgress	CurrentFileIndex, TotalFiles, CurrentFileProgress, CurrentFileName	Update one bar/label; PS1 calculates percent from bytes.
StartPost	PostText, SelectedMedia	Freeze request, export first, start Python, handle completion.
ExportPostRecord	PostText, SelectedMedia	Create TXT/media copies/history row; return PostRecordPath.
RunPythonEngine	PostText, SelectedMedia / archived paths	Start nonblocking Python, stream JSONL, stderr, exit.
HandlePythonResponse	PythonMessage	Validate correlation/sequence and dispatch event or synthetic failure.

7.1 Locked shared state names

```
Config; StartupMode; PostText; SelectedMedia; PostInProgress
CurrentFileIndex; TotalFiles; CurrentFileProgress; CurrentFileName
PostRecord; PostRecordPath
PythonProcess; PythonMessage; PythonResult
PublishedPostId; PublishedPostUrl
RepostData; RepostTextPath; RepostMedia
```

Additional transport/handshake internal state may be introduced with readable names; the locked names above must not be casually renamed. PS1 may implement helpers for timer, safe process queue, parser, watchdog, CSV updates and logging without turning them into extra application controllers.

8. JSON-Lines posting protocol

8.1 Transport and framing

PowerShell starts Python directly (UseShellExecute=false), passing --config and a posting mode. PowerShell writes ONE complete compact JSON request as UTF-8 without BOM to stdin, flushes and closes stdin. Python stdout contains ONLY complete JSON objects, one per physical line, flushed immediately; stderr is reserved for raw/unexpected diagnostics. Python JSON output may ASCII-escape non-ASCII characters (ensure_ascii=True) while preserving the original Unicode text. Do not use an unsupported Windows PowerShell 5.1 StandardInputEncoding setter; use a proven byte-safe stdin method. No HTTP localhost service or request/response temporary transport files.

8.2 Complete request example

```
{"ProtocolVersion":1,"RequestId":"req-example-001","Text":"Example post","TotalFiles":2,"Media":
[{"MediaId":1,"FileIndex":1,"FileName":"photo1.jpg","FilePath":"Z:\\...\\photo1.jpg"},
{"MediaId":2,"FileIndex":2,"FileName":"photo2.jpg","FilePath":"Z:\\...\\photo2.jpg"}]}
```

All Media[] objects are present in the single request. FileName is generic (never FileName1/FileName2). Local MediaId and remote XMediaId are different identities. PostId and XMediaId are strings, never floating-point numbers.

8.3 Exactly eight posting events

Event	Purpose / essential fields
READY	RequestId, ProtocolVersion; request and required local inputs validated.
TEXT_COMPLETE	RequestId, TextPresent, CharacterCount; text prepared, NOT published.
PROGRESS	RequestId, MediaId, FileIndex, TotalFiles, FileName, BytesUploaded, BytesTotal; NO Percent field.
MEDIA_COMPLETE	RequestId, MediaId, FileIndex, TotalFiles, FileName, XMediaId; media ready after X processing.
POSTING	RequestId, MediaCount; final create-post attempt is about to begin.
DEBUG	RequestId, Source="PYTHON", Level, Stage, Message, optional MediaId/FileIndex; nonterminal.
SUCCESS	RequestId, PostId, PostUrl; terminal.
ERROR	RequestId when recoverable, Code, Stage, Message, Retryable; optional MediaId/FileIndex/HttpStatus; terminal.

```
READY → [DEBUG]* → TEXT_COMPLETE → [DEBUG]*
      → (PROGRESS* → MEDIA_COMPLETE) repeated for each media
      → POSTING → [DEBUG]* → SUCCESS
At any nonterminal point: ERROR (terminal)
No stdout protocol events after SUCCESS or ERROR.
```

DEBUG can occur between valid nonterminal events without advancing the posting state. If a malformed request has no recoverable RequestId, Python may emit one best-effort ERROR with null RequestId; PowerShell accepts this exception ONLY for the pre-READY malformed-request case. Controlled DEBUG messages are written through Write-DebugLine with a [PYTHON] prefix and displayed in the permanent Debug/Output window. stderr is never parsed as posting JSON.

8.4 Byte-based progress — latest locked change

```
{"Event":"PROGRESS","RequestId":"req-
example-001","MediaId":1,"FileIndex":1,"TotalFiles":2,"FileName":"photo1.jpg","BytesUploaded":4194304,"BytesTotal":
10485760}

PS1: CurrentFileProgress = clamp(100 * BytesUploaded / BytesTotal, 0, 100)
Example: 4194304 / 10485760 × 100 = 40%
```

Python reports only ACTUAL confirmed bytes/chunks, never a fabricated percentage. PS1 owns the arithmetic and GUI display. A completed byte transfer may show 100% while X is still processing; status must say "Processing media" until MEDIA_COMPLETE. A non-progress heartbeat, when needed for watchdog liveness, uses DEBUG/status information rather than fabricated PROGRESS byte changes. For zero-byte media, validation fails before division.

8.5 Protocol state machine

Current phase	Allowed event	Next phase / checks
Starting	READY	Ready; validate RequestId/version.
Starting / Ready / Media / Posting	ERROR	Terminal/Draining; stop normal flow.
Ready	TEXT_COMPLETE	Media; verify TextPresent and request correlation.
Media	PROGRESS	Media; only current MediaId/FileIndex; nondecreasing bytes ≤ total.
Media	MEDIA_COMPLETE	Media; verify XMediaId; advance current media index.
Media, all items complete	POSTING	Posting; MediaCount matches.
Posting	SUCCESS	Terminal/Draining; PostId and PostUrl present.
Any nonterminal phase after request correlation	DEBUG	Same phase; validate Source and safe fields.
Terminal / Draining	Any further protocol event	Violation; never treat as another result.

For a text-only post, the Media phase contains no upload events and POSTING follows TEXT_COMPLETE. For media-only, TEXT_COMPLETE is still emitted with TextPresent=false. Wrong RequestId, malformed JSON, invalid ordering, duplicate terminal event, unknown event or missing fields are protocol failures. Process exit without a valid terminal event is a PowerShell synthetic failure.

9. Python engine workflow and function contracts

9.1 Posting-mode sequence

```
Main(--config, post mode)
  → ReadRequest(stdin) / extract RequestId as early as possible
  → ReadConfig / ValidateConfig
  → ValidateRequest / ValidateMediaItem
  → EmitEvent READY
  → PrepareText → EmitEvent TEXT_COMPLETE
  → LoadCredentials / ValidateCredentials / AuthenticateX
  → UploadMediaCollection (0..4, sequential)
       → UploadMedia for each file
       → EmitDebug / EmitEvent PROGRESS (confirmed bytes)
       → EmitEvent MEDIA_COMPLETE after processing
  → EmitEvent POSTING
  → CreatePost (one final X request)
  → EmitEvent SUCCESS → flush → exit 0

Any failure → ClassifyError → EmitError → flush → exit nonzero
```

No media upload or X post begins before the full local request has been validated. PrepareText preserves text, Unicode and line breaks and does not rewrite it. Python never creates local history, modifies the GUI, chooses a Title, or initiates Repost.

9.2 Python functions

Function	Input → output / responsibility
Main	ConfigPath + stdin or health mode → orchestrated exit; one operation per process.
ReadConfig / ValidateConfig	ConfigPath → Python-relevant settings; no duplicated hard-coded paths.
ReadRequest / ValidateRequest	one JSON line → validated complete request; check ProtocolVersion, RequestId, Text, TotalFiles, Media length and content.
ValidateMediaItem	media + expected index → valid local MediaId/FileIndex, readable path, verified media constraints.
PrepareText	Text → same content validated; TEXT_COMPLETE emitted by Main.
LoadCredentials / ValidateCredentials	KeysFilePath → OAuth2 fields; no secret output.
AuthenticateX	credentials + config → authenticated X user-context client; verified

Function	Input → output / responsibility
	refresh behavior.
UploadMediaCollection	XClient + Media[] + RequestId → ordered XMediaIds[]; abort on first failure.
UploadMedia	one media → remote XMediaId; real byte events, finalize and processing checks.
CreatePost	prepared text + XMediaIds[] → PostId/PostUrl; never auto-resend.
EmitEvent	single stdout writer; compact JSONL + immediate flush.
EmitDebug	nonterminal DEBUG event; Source=PYTHON, safe fields.
ClassifyError / EmitError	stable error code/stage/message → terminal ERROR.
WriteRawDiagnostic	rare unexpected stderr diagnostic; redact secrets.
ExitApplication	0 on SUCCESS, nonzero on ERROR; no post-terminal stdout.

9.3 Request validation rules

ProtocolVersion=1; RequestId non-empty; Text field present and a string (possibly empty); TotalFiles integer 0..4; Media array length equals TotalFiles; Text or at least one media must exist; MediaId unique; FileIndex sequential 1..TotalFiles; FileName present; FilePath absolute and readable. Reject inconsistencies explicitly; never silently repair an invalid request. X media type/size/duration and text-length checks must use current verified API constraints rather than obsolete hard-coded assumptions.

10. X authentication, media upload, and create-post

10.1 Selected OAuth approach

Use OAuth 2.0 User Context with the chosen X_Keys.txt fields and the scopes required by the selected current endpoints. AuthenticateX owns access-token validity and refresh. The application does not use the normal X username/password/Google Authenticator code during API posting. Verify actual X app/client configuration, callback/refresh-token handling, endpoint authorization, API tier, and account billing before live use.

10.2 Candidate X API v2 upload sequence — verify against current documentation

```
For each media file (sequential):
  INITIALIZE  POST /2/media/upload/initialize
      → receive remote XMediaId
  APPEND      POST /2/media/upload/{id}/append
      → send chunk, await acknowledgment, update BytesUploaded
  FINALIZE    POST /2/media/upload/{id}/finalize
      → may return processing state
  STATUS      GET /2/media/upload?command=STATUS&media_id={id}
      → poll if required until succeeded or failed
  MEDIA_COMPLETE (only after media usable)
After all files: POST /2/tweets → PostId / PostUrl
```

These paths are inherited from Person 2 as an engineering candidate, not proof of live OAuth2/media compatibility. Confirm current endpoint signatures, media limits, required scopes, supported OAuth2 token type, billing and processing response formats with official X documentation and a controlled real upload. Do not substitute old v1.1 endpoints or claim the upload has been live-tested. Do not confuse the remote XMediaId returned at INITIALIZE with final MEDIA_COMPLETE readiness.

10.3 Partial failure and unknown create result

If media 1 succeeds but media 2 fails, Python emits ERROR and stops: no POSTING, no SUCCESS, no final post. Partial XMediaIds are attempt-local diagnostics and are not reused automatically. If final create-post response is lost after a request may have reached X, report POST_RESULT_UNKNOWN and require the user to check X before any new attempt. Never automatically resend an uncertain final post.

11. Nonblocking PowerShell process management

11.1 Safe asynchronous event pump

Preserve Person 2's correction for Windows PowerShell 5.1: do not attach arbitrary PowerShell script-block callbacks to .NET process DataReceived events on thread-pool threads and do not directly update WinForms from those threads. Use a safe process reader/queue or equivalent runspace-safe transport. A GUI-thread pump (Person 2 proposed a 100 ms WinForms timer) drains queued stdout/stderr/exit messages and calls HandlePythonResponse on the UI thread. This 100 ms pump is NOT the 10-second idle handshake timer.

11.2 Watchdogs and exit semantics

Watchdog	Person 2 proposed default	Required handling
Startup	30 seconds	No READY → controlled timeout/synthetic failure.
Idle posting output	90 seconds	No protocol/debug liveness → controlled timeout; do not invent progress.
Total posting	30 minutes	Terminate safely; Posting-phase uncertainty triggers manual verification.
Terminal drain grace	5 seconds	Drain streams/exit; no second result or hidden retry.
Health-check process	Shorter than 10-second interval (verify)	Skip overlap, return reason on timeout, no GUI freeze.

The numerical posting timeouts above are inherited engineering defaults, NOT user-locked values; validate them for real media/video processing before live use. A timeout during the Posting phase is potentially an unknown remote outcome. After a terminal event, drain stdout/stderr and observe process exit; process exit alone is not proof of SUCCESS.

11.3 Stable errors and security

Python-controlled errors include MALFORMED_REQUEST_JSON, INVALID_REQUEST, EMPTY_POST, CONFIG_ERROR, CREDENTIAL_ERROR, AUTH_FAILED, RATE_LIMITED, NETWORK_ERROR, MEDIA_UPLOAD_FAILED, POST_CREATE_FAILED, POST_RESULT_UNKNOWN and INTERNAL_ERROR. PowerShell-synthesized failures include PYTHON_START_FAILED, PYTHON_EXITED_UNEXPECTEDLY, PROTOCOL_SEQUENCE_ERROR, malformed stdout, RequestId mismatch and OPERATION_TIMEOUT. Preserve stable Stage, Code, safe Message, Retryable, and optional media/HTTP context. Do not treat Retryable=true as authorization for an automatic post retry.

12. Idle handshake — independent 10-second health-check plan

12.1 Locked ownership and idle-only rule

```
GUI IDLE / PostInProgress = false
  → PS1 timer reaches 10 seconds
  → if no prior handshake active: start short-lived Python health mode
  → Python loads config/credentials and performs read-only X check
  → Python returns HEALTH_CHECK result and exits
  → PS1 handles response on GUI thread
  → repeat while idle

POSTING: suspend timer; never start new health check.
POST FINISHED (success or failure): resume 10-second idle schedule.
```

PowerShell owns WHEN the check runs, Python owns HOW X is checked, and PowerShell owns WHAT the user sees and logs. Python does not own a recurring timer. Only one handshake may be active at once; skip a tick if the previous check is unfinished. An in-flight handshake when Post is pressed must be safely completed or stopped without delaying or interfering with publication; do not launch overlapping X operations from the handshake path.

12.2 Health response contract — separate from posting events

```
SUCCESS (illustrative):
{"Event":"HEALTH_CHECK","Status":"SUCCESS"}
FAILURE (illustrative):
{"Event":"HEALTH_CHECK","Status":"FAILURE","Reason":"NETWORK_ERROR","Message":"Unable to reach X"}
```

HEALTH_CHECK is a separate health-mode response, NOT a ninth posting protocol event. The exact read-only X endpoint, OAuth2 permissions, rate-limit behavior, request cost, and timeout must be verified before coding. A 10-second cadence can create a substantial number of requests; do not claim the endpoint is free or exempt from rate limits. An HTTP 404 from an unauthenticated API root proves only HTTPS reachability, not authenticated account health. A successful health check must reflect the selected authenticated endpoint's actual response.

12.3 User-visible behavior and logging

Condition	Debug / Output window	Existing DebugLog.ps1
Health check succeeds	Silent	No routine success entry.
First failure	Handshake failed (<reason>)	Write-DebugLine with [PYTHON] source and safe reason.
Repeated identical failure	No duplicate warning	No repeated identical warning.
Different failure reason	Show new warning	Log new warning.
Connection recovers	Silent; reset failure suppression	No routine success entry.
Later new failure	Show warning again	Log warning again.

Failure categories may include NETWORK_ERROR, DNS_ERROR, TIMEOUT, AUTH_FAILED, RATE_LIMITED, X_SERVICE_ERROR, INTERNAL_ERROR, malformed response, and Python process start/exit errors. A handshake failure does not close the GUI, erase content, disable Post permanently, upload media, or publish anything. For Python-reported X failures use the [PYTHON] message prefix; for a PowerShell-side Python launch or transport failure use [PS1] while preserving the same user-visible Handshake failed (<reason>) wording. Avoid logging access tokens or user post contents. The timer and any active child process must be cleaned up on GUI close.

12.4 Handshake test matrix

Test	Expected
Idle for multiple intervals	One check approximately every 10 seconds; never overlapping.
Healthy X endpoint	No GUI/debug-log success chatter.
First network/auth failure	One Handshake failed (<reason>) warning in GUI and logger.
Same failure persists	No repetitive warnings every 10 seconds.
Reason changes	New warning logged/displayed.
Recovery and later outage	Silent recovery resets suppression; new outage warned.
Post starts	No new health checks; posting takes priority.
Post ends successfully or fails	Idle checks resume.
Slow/hung Python or invalid JSON	Timeout/error safely reported; GUI responsive.
Close GUI	Timer stops and child processes cleaned up.

13. Full acceptance and test plan

13.1 Mock-first verification

Retain Person 2's Python mock backend, fault injection and conformance harness as engineering assets. Person 2 reported 22/22 Python mock conformance tests passed in its environment; that does NOT establish that PowerShell has been executed on the user's machine or that any live X API call succeeded. Re-run the harness after replacing Percent with BytesUploaded/BytesTotal, adding DEBUG and health mode, and integrating the actual logger and archive. Record real test evidence separately.

ID	Scenario	Acceptance criterion
APP-01	Text-only post	Post enabled; local TXT/history first; READY→TEXT_COMPLETE→POSTING→SUCCESS.
APP-02	Media-only post	TEXT_COMPLETE TextPresent=false; sequential media; success.
APP-03	Both empty	Post disabled; Python not launched.
APP-04	Attach/Remove up to four	State and Post button update; no premature upload.
APP-05	Export fails	Python never starts; visible error and preserved user content.
APP-06	X fails after export	Local record remains, PostURL/Timestamp blank.
APP-07	X succeeds	Same history row updated with actual PostURL and publication time.
APP-08	Repost JSON	Preloads TXT/media; no automatic Post; user may edit.
APP-09	Byte progress	PS1 computes percentage; per-file switching; no Python Percent.
APP-10	Media processing	Transfer 100% does not imply MEDIA_COMPLETE.
APP-11	DEBUG and stderr	Structured DEBUG logged [PYTHON]; stderr raw only.
APP-12	Wrong RequestId/event order	Protocol error, no silent success.
APP-13	Partial upload failure	No POSTING or SUCCESS.
APP-14	Uncertain post result	No auto retry; explicit manual reconciliation.
APP-15	Unicode/newlines	Post text preserved through JSON, archive and engine.
APP-16	Process crash/hang	Synthetic error; GUI responsive; correct cleanup.
APP-17	Idle handshake	Silent success; deduplicated failure; pause/resume; no overlap.
APP-18	Secret scan	No credential values in stdout, stderr, GUI or logs.
APP-19	CSV/history	Eight exact columns; clickable Excel display and safe path round-trip.
APP-20	Live smoke test	OAuth2 authenticated call, one text-only post, one media post, real URL.
APP-21	PowerShell 5.1 compatibility	No invalid encoding setters/runspace callback failures.
APP-22	GUI shutdown	No leaked timer/process; safe unknown-outcome warning if applicable.

13.2 Failure and concurrency matrix

Mock and fault-injection scenarios must include malformed JSON, missing file, duplicate MediaId, bad FileIndex, unknown event, duplicate terminal event, wrong RequestId, auth failure, rate limit, media failure after a prior media success, posting timeout, Python startup failure, stdout corruption, stderr traceback, token refresh failure, health-check timeout, health-check 429, and simultaneous Post click while a handshake is active.

14. Phased implementation work plan and release gates

Phase	Work package	Exit gate
0. Source lock	Approve this specification and exact interfaces.	No unresolved design conflicts or accidental old defaults.
1. Stage 1 real communication	Verify developer/billing status, OAuth2, authenticated X call and one demo post.	Actual authenticated demo post through X API; do not claim completed from HTTPS ping.
2. Protocol and Python mock	Request parser, 8-event JSONL, byte progress, DEBUG, mock failures.	Python conformance harness passes revised contract.
3. PS1 transport/GUI	WinForms, process queue, UI pump, state machine, logger.	Target-machine PowerShell 5.1 run and responsive fake posting.
4. Local archive and Repost JSON	TXT, media copies, exact CSV row, Title, startup preload.	Export-before-Python, safe history updates, manual Repost.
5. Idle handshake	PS1 timer, Python health mode, warnings/dedup/pause/resume.	All handshake acceptance tests pass.
6. Real X backend	OAuth2 refresh, verified v2 upload, post creation, real byte reporting.	One text and one media live smoke test; safe failures.
7. Integration/hardening	Unicode, unknown outcome, timeouts, cleanup, security and CSV.	Full APP matrix and three independent code reviews.
8. Deferred Excel template	Design workbook and Repost invocation after JSON contract proves stable.	User separately approves workbook/launch details.

The earlier Stage 1 gate remains a real-posting prerequisite; the existence of this design document does not waive it. Documentation and mock testing may be prepared before the gate, but do not represent the application as live-ready until the authenticated demo post and subsequent integration gates pass.

15. Explicitly deferred or verification-required items

Item	Disposition
Excel workbook/Repost button invocation	DEFERRED; application consumes validated JSON; workbook design later.
Current X endpoint syntax and OAuth2 media support	VERIFY BEFORE LIVE USE using current official X documentation.
OAuth2 token refresh, client configuration and scopes	VERIFY BEFORE LIVE USE with actual account; no secrets in documents.
Media type/size/duration/text constraints	VERIFY BEFORE LIVE USE; avoid obsolete Person 2 assumptions.
Health-check endpoint, rate limits, request cost	VERIFY BEFORE LIVE USE; every 10 seconds must be sustainable.
CSV hyperlink implementation and safe Excel round-trip	VERIFY during history integration; retain eight-column user-visible design.
Posting/health timeout values	ENGINEERING DEFAULTS; tune only with measured tests.
XML/XSL integration, scheduling, bookmarks, replies, quotes	OUT OF SCOPE unless separately approved.
Production source-code replacement	NOT performed by this document-generation assignment.

16. Developer handoff and definition of done

The implementer must use this unified specification rather than copying either original PDF unchanged. The target is a readable, personal-use application with centralized configuration, shared logging, explicit functions, and real-time diagnostics. For code work, first load the user's active coding protocol, explain logic before coding, and obtain approval. Every meaningful failure must be traceable to a function, cause and fix; do not silently repair malformed protocol messages or mutate user text.

DEFINITION OF DONE: GUI and archive behavior match this document; the eight-event JSONL contract is enforced; Python reports byte counts while PS1 calculates percentages; OAuth2/live endpoints have passed real tests; the independent idle handshake behaves exactly as locked; history/Repost boundaries are respected; no duplicate posts are caused by automatic retry; the target-machine PowerShell and Python tests pass; logs are safe; and unresolved/deferred items are not falsely described as complete.